

```
1: #include "esp_camera.h"
2: #include "esp_wifi.h"
3: #include "freertos/FreeRTOS.h"
4: #include "freertos/semphr.h"
5: #include <stdarg.h>
6: #include <WiFi.h>
7:
8: #ifndef BUZZER_PIN
9: #define BUZZER_PIN 2
10: #endif
11:
12: // =====
13: // WLAN-ZUGANG
14: // =====
15: const char *wlan_name = "FRITZ!Box 7530 NV";
16: const char *wlan_passwort = "Donner1997";
17:
18: // =====
19: // PINBELEGUNG FUER FREENOVE ESP32-S3 (0V3660)
20: // =====
21: #define PWDN_GPIO_NUM -1
22: #define RESET_GPIO_NUM -1
23: #define XCLK_GPIO_NUM 15
24: #define SIOD_GPIO_NUM 4
25: #define SIOC_GPIO_NUM 5
26:
27: #define Y9_GPIO_NUM 16
28: #define Y8_GPIO_NUM 17
29: #define Y7_GPIO_NUM 18
30: #define Y6_GPIO_NUM 12
31: #define Y5_GPIO_NUM 10
32: #define Y4_GPIO_NUM 8
33: #define Y3_GPIO_NUM 9
34: #define Y2_GPIO_NUM 11
35:
36: #define VSYNC_GPIO_NUM 6
37: #define HREF_GPIO_NUM 7
38: #define PCLK_GPIO_NUM 13
39:
40: // Wird aus app_httpd.cpp bereitgestellt.
41: void startCameraServer();
42:
43: // Diese Namen bleiben gleich, weil app_httpd.cpp sie per extern erwartet.
44: volatile uint32_t wifiDisconnectCount = 0;
```

```
45: volatile uint32_t wifiGotIpCount = 0;
46: volatile uint32_t buzzerPulseUntilMs = 0;
47:
48: static SemaphoreHandle_t fern_log_mutex = NULL;
49: static String fern_log_puffer;
50: static const size_t FERN_LOG_MAX_BYTES = 16384;
51:
52: static void fern_log_anhaengen_gesperrt(const char *nachricht) {
53:     fern_log_puffer += nachricht;
54:     size_t ueberlauf =
55:         fern_log_puffer.length() > FERN_LOG_MAX_BYTES
56:         ? fern_log_puffer.length() - FERN_LOG_MAX_BYTES
57:         : 0;
58:     if (ueberlauf > 0) {
59:         fern_log_puffer.remove(0, ueberlauf);
60:     }
61: }
62:
63: void fern_log_puffer_initialisieren() {
64:     if (fern_log_mutex == NULL) {
65:         fern_log_mutex = xSemaphoreCreateMutex();
66:     }
67: }
68:
69: // Namen bleiben gleich fuer die Kopplung mit app_httpd.cpp.
70: void remoteLogf(const char *format, ...) {
71:     char nachricht[256];
72:     va_list argumente;
73:     va_start(argumente, format);
74:     vsnprintf(nachricht, sizeof(nachricht), format, argumente);
75:     va_end(argumente);
76:
77:     Serial.print(nachricht);
78:
79:     if (fern_log_mutex != NULL &&
80:         xSemaphoreTake(fern_log_mutex, pdMS_TO_TICKS(50)) == pdTRUE) {
81:         fern_log_anhaengen_gesperrt(nachricht);
82:         xSemaphoreGive(fern_log_mutex);
83:     }
84: }
85:
86: void remoteLogln(const char *nachricht) {
87:     Serial.println(nachricht);
88: }
```

```
89:   if (fern_log_mutex != NULL &&
90:       xSemaphoreTake(fern_log_mutex, pdMS_TO_TICKS(50)) == pdTRUE) {
91:       fern_log_anhaengen_gesperrt(nachricht);
92:       fern_log_anhaengen_gesperrt("\n");
93:       xSemaphoreGive(fern_log_mutex);
94:   }
95: }
96:
97: String getRemoteLogSnapshot(bool nach_lesen_leeren) {
98:   String schnappschuss;
99:   if (fern_log_mutex != NULL &&
100:      xSemaphoreTake(fern_log_mutex, pdMS_TO_TICKS(100)) == pdTRUE) {
101:     schnappschuss = fern_log_puffer;
102:     if (nach_lesen_leeren) {
103:       fern_log_puffer = "";
104:     }
105:     xSemaphoreGive(fern_log_mutex);
106:   }
107:   return schnappschuss;
108: }
109:
110: void bei_wlan_ereignis(WiFiEvent_t ereignis, WiFiEventInfo_t info) {
111:   switch (ereignis) {
112:     case ARDUINO_EVENT_WIFI_STA_DISCONNECTED:
113:       wifiDisconnectCount++;
114:       remoteLogf("[WIFI] disconnected reason=%u total=%lu\n",
115:                  info.wifi_sta_disconnected.reason,
116:                  (unsigned long)wifiDisconnectCount);
117:       break;
118:     case ARDUINO_EVENT_WIFI_STA_GOT_IP:
119:       wifiGotIpCount++;
120:       remoteLogf("[WIFI] got_ip total=%lu ip=%s\n",
121:                  (unsigned long)wifiGotIpCount,
122:                  WiFi.localIP().toString().c_str());
123:       break;
124:     default:
125:       break;
126:   }
127: }
128:
129: void setup() {
130:   Serial.begin(115200);
131:   Serial.setDebugOutput(false);
132:   fern_log_puffer_initialisieren();
```

```
133:   Serial.println();
134:
135:   pinMode(BUZZER_PIN, OUTPUT);
136:   digitalWrite(BUZZER_PIN, LOW);
137:
138:   camera_config_t kamera_konfiguration;
139:   kamera_konfiguration.ledc_channel = LEDC_CHANNEL_0;
140:   kamera_konfiguration.ledc_timer = LEDC_TIMER_0;
141:   kamera_konfiguration.pin_d0 = Y2_GPIO_NUM;
142:   kamera_konfiguration.pin_d1 = Y3_GPIO_NUM;
143:   kamera_konfiguration.pin_d2 = Y4_GPIO_NUM;
144:   kamera_konfiguration.pin_d3 = Y5_GPIO_NUM;
145:   kamera_konfiguration.pin_d4 = Y6_GPIO_NUM;
146:   kamera_konfiguration.pin_d5 = Y7_GPIO_NUM;
147:   kamera_konfiguration.pin_d6 = Y8_GPIO_NUM;
148:   kamera_konfiguration.pin_d7 = Y9_GPIO_NUM;
149:   kamera_konfiguration.pin_xclk = XCLK_GPIO_NUM;
150:   kamera_konfiguration.pin_pclk = PCLK_GPIO_NUM;
151:   kamera_konfiguration.pin_vsync = VSYNC_GPIO_NUM;
152:   kamera_konfiguration.pin_href = HREF_GPIO_NUM;
153:   kamera_konfiguration.pin_sccb_sda = SIOD_GPIO_NUM;
154:   kamera_konfiguration.pin_sccb_scl = SIOC_GPIO_NUM;
155:   kamera_konfiguration.pin_pwdn = PWDN_GPIO_NUM;
156:   kamera_konfiguration.pin_reset = RESET_GPIO_NUM;
157:   kamera_konfiguration.xclk_freq_hz = 15000000;
158:   kamera_konfiguration.frame_size = FRAMESIZE_VGA;
159:   kamera_konfiguration.pixel_format = PIXFORMAT_JPEG;
160:   kamera_konfiguration.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
161:   kamera_konfiguration.fb_location = CAMERA_FB_IN_PSRAM;
162:   kamera_konfiguration.jpeg_quality = 16;
163:   kamera_konfiguration.fb_count = 1;
164:
165:   if (psramFound()) {
166:     kamera_konfiguration.jpeg_quality = 14;
167:     kamera_konfiguration.fb_count = 2;
168:     kamera_konfiguration.frame_size = FRAMESIZE_VGA;
169:     kamera_konfiguration.grab_mode = CAMERA_GRAB_LATEST;
170:     remoteLogln("psramFound");
171:   } else {
172:     kamera_konfiguration.frame_size = FRAMESIZE_VGA;
173:     kamera_konfiguration.fb_location = CAMERA_FB_IN_DRAM;
174:   }
175:
176:   esp_err_t fehler = esp_camera_init(&kamera_konfiguration);
```

```
177:   if (fehler != ESP_OK) {
178:       remoteLogf("Kamera-Init fehlgeschlagen: 0x%x\n", fehler);
179:       return;
180:   }
181:
182:   sensor_t *kamera_sensor = esp_camera_sensor_get();
183:   if (kamera_sensor->id.PID == OV3660_PID) {
184:       kamera_sensor->set_vflip(kamera_sensor, 1);
185:       kamera_sensor->set_brightness(kamera_sensor, 1);
186:       kamera_sensor->set_saturation(kamera_sensor, -2);
187:   }
188:
189:   WiFi.onEvent(bei_wlan_ereignis);
190:   WiFi.mode(WIFI_STA);
191:   WiFi.begin(wlan_name, wlan_passwort);
192:   WiFi.setSleep(false);
193:
194:   while (WiFi.status() != WL_CONNECTED) {
195:       delay(500);
196:       Serial.print(".");
197:   }
198:   remoteLogln("WiFi verbunden!");
199:   esp_wifi_set_ps(WIFI_PS_NONE);
200:   remoteLogln("WLAN Power Management deaktiviert fuer stabilen Stream");
201:
202:   startCameraServer();
203:
204:   Serial.print("Kamera bereit! Oeffne: http://");
205:   Serial.println(WiFi.localIP());
206: }
207:
208: void loop() {
209:   uint32_t buzzer_ende_zeit = buzzerPulseUntilMs;
210:   if (buzzer_ende_zeit != 0 && (int32_t)(millis() - buzzer_ende_zeit) >= 0) {
211:       digitalWrite(BUZZER_PIN, LOW);
212:       buzzerPulseUntilMs = 0;
213:   }
214:
215:   delay(5);
216: }
```